

Question **1**

Answer saved

Marked out of 4.00

🚩 Flag question

Consider the following partial class declaration of an ADT:

```
class Employee

{

    friend ostream& operator << (ostream& outs,  const Employee & the_emp);  //output name and salary with salary in the format Rxxx.xx

public:

    Employee(string n, double sal);

    ~Employee();

    void setSalary(double sal);

    string getName()const;

    void raiseSalary(double percent); // raises an employee's salary with the percentage specified by the argument percent

private:

    string name;

    double salary;

};
```

Complete the code below by choosing the correct options in the drop-down lists. The comment with each drop-down list describes the action that the option you choose from the drop-down list should achieve.

```
Employee newEmployee ("Mpho Malebane", 123000); // Use the overloaded constructor to instantiate an object newEmployee of
the class Employee with the name "Mpho Malebane"

//and a salary of R123 000

if (newEmployee.getName() == "Mpho Malebane") // Write a statement to check whether the object newEmployee is Mpho Malebane
newEmployee.raiseSalary(10); // Raise Mpho Malebane's salary with 10%

cout << newEmployee << endl; // Use the overloaded stream insertion operator to display
the name and salary of the newEmployee on the screen
```

Question **2**

Answer saved

Marked out of 1.00

🚩 Flag question

Given the declarations

```
string *p1;

int arraySize;
```

The correct code to declare a dynamic array of strings (using the string pointer variable p1) that has arraySize elements is

p1 = new string[arraySize];

Question **3**

Answer saved

Marked out of
1.00

🚩 Flag question

To confirm whether an integer n is positive, we can use the following statement

Select one:

- ☐ `assert(n < 0);`
- ☐ `assert(!(n > 0));`
- ☒ `assert(n > 0);`
- ☐ None of the other options are valid.

[Clear my choice](#)

Question **4**

Answer saved

Marked out of
1.00

🚩 Flag question

The file that contains the definition of the class is called the

- ☒ interface file
- ☐ the application file
- ☐ the implementation file
- ☐ the specification file

[Clear my choice](#)

Question **5**

Answer saved

Marked out of 1.00

🚩 Flag question

When would you want to make a function a function template?

Select one:

- ☐ All functions should be function templates.
- ☒ When the implementation details of the function are independent of the data type(s) of the parameters.
- ☐ Only when two functions have the same type of parameters.
- ☐ When two different functions have different implementation details.

Clear my choice

Question **6**

Answer saved

Marked out of 8.00

🚩 Remove flag

For this question you have to type code in the textbox provided. You cannot attach a file as answer. NB: do not use TAB when entering your code, rather use spaces to indent.

Consider the following (partial) class declaration of an ADT provided in a file `Player.h`:

```
class Player
{
    friend bool operator > (const Player &player1, const Player &player2); // returns true if player1 has more points than player2, false otherwise

    friend ostream & operator<<(ostream & out, const Player & P); //displays player's name and points

public:
    Player();
    ~Player();
    string getName() const;
    void setPoints(int p);

private:
    string name;
    int points;
};
```

Using **separate compilation**, provide the **implementation file** for the class `Player`. Include all the necessary header files, and implement **only** the member functions in the given class declaration. There is no need for comments.

↴

A ▾

B

I

☰

☷

☰

☷

🔗

🔄

😊

🖼️

```
#include <iostream>
#include <string>
#include "Player.h"

using namespace std;

bool operator > (const Player &player1, const Player &player2) {
    return player1.points > player2.points;
};

ostream& operator <<(ostream& out, const Player &P) {
    out
        << "Player name: " << P.name
        << "Player points: " << P.points
    ;
    return out;
};

Player::Player():
```

Question **7**

Answer saved

Marked out of
1.00

🚩 Flag question

Which of the following declarations correctly creates a C-string `s1` that can hold the value "phonebook"

Select one:

- ☐ `char s1;`
- ☐ `char s1 = 10;`
- ☐ `char s1[9];`
- ☒ `char s1[10];`

Clear my choice

Question **8**

Answer saved

Marked out of
1.00

🚩 Flag question

What is the difference between `strcmp` and `strncmp`?

Select one:

- ☒ They both compare C-strings, one expects an integer for the number of characters to compare
- ☐ They are in different libraries
- ☐ One copies C-strings, the other compares C-strings
- ☐ No difference

Clear my choice

Question **9**

Answer saved

Marked out of
1.00

🚩 Flag question

Automatic variables are automatically created in the heap or freestore when they are declared and automatically released when the function where they are declared ends.

Select one:

☐ True☒ FalseQuestion **10**

Answer saved

Marked out of
1.00

🚩 Flag question

Vectors resemble dynamic arrays because the number of elements in both a vector and a dynamic array can vary during program execution.

Select one:

☒ True☐ False

Question **11**

Answer saved

Marked out of 1.00

🚩 Flag question

Given the code

```
vector<int> v;
```

```
int elem;
```

to add `elem` to `v`, we use the function

v.push_back(elem); ⚡

Question **12**

Answer saved

Marked out of 1.00

🚩 Flag question

Consider the following function definition:

```
int surprise(int x, int y)
{
    return x + y;
}
```

To turn function `surprise` into a template function, we need to change it as follows:

Select one:

☐ `template <T>`

```
T surprise(T x, T y)
{
    return x + y;
}
```

☐ `template <class T1, class T2>`

```
T1 surprise(T1 x, T2 y)
{
    return T1 + T2;
}
```

☐ `Template <class T>`

```
T surprise(T x, T y)
{
    return x + y;
}
```

☒ `template <class T>`

```
T surprise(T x, T y)
{
    return x + y;
}
```

Clear my choice

Question **13**

Answer saved

Marked out of 1.00

Flag question

Consider the following function headers:

```
double calcVolume (double width, double height, double length);
double calcVolume (double radius);
double calcVolume (double radius, double height);
```

The function call

```
cout << calcVolume(7.3, 5.1);
```

will execute this function

Select one:

- ☐ double calcVolume (double radius);
- ☐ It will give a compiler error.
- ☐ double calcVolume (double width, double height, double length);
- ☒ double calcVolume (double radius, double height);

Clear my choice

Question **14**

Not yet answered

Marked out of 8.00

Remove flag

For this question you have to type code in the textbox provided. You cannot attach a file as answer. NB: do not use TAB when entering your code, rather use spaces to indent.

Use **separate compilation** to **declare a class** `Player` as an **ADT**. Remember to include the necessary directives.

The class `Player` has two member variables,

- `name`, the name of the player (a string) and
- `points`, the number of points accumulated (an integer).

Provide only the class declaration (interface) for the class `Player`. The class `Player` has the following member functions:

- an overloaded constructor to initialise the member variables to specific values
- a destructor
- an accessor for the member variable `points`
- a mutator for the member variable `name`
- a member function `increment()` that increments the points by a value passed as parameter
- an overloaded stream extraction operator for the class `Player`.

↴

A ▾

B

I

≡

≡

≡

≡

🔗

🔗

😊

🖼️

Question **15**

Answer saved

Marked out of 1.00

🚩 Flag question

What is the proper way to declare a vector of strings called names?

- Select one:
- ☒ `vector<string> names;`
 - ☐ `vector strings names;`
 - ☐ `vector<names> string;`

Clear my choice

Question **16**

Answer saved

Marked out of 8.00

🚩 Flag question

Consider the partially complete program to read learner’s names and grades from a file called "Names.dat".

The program then displays the names of all learners in Grade 7 on the console.

Complete the code below by choosing the correct options in the drop-down lists.

The comment next to each drop-down list describes the action that the option you choose from the drop-down list should achieve. The comments in between the statements indicate code that would complete the program.

```
#include < 

fstream

 > //directive to use files in a program

//other program directives

using namespace std;

int main()

{



ifstream inFile;

 //declare input stream for a file

int grade;

string name;



inFile.open("Names.dat");

 //open and connect input stream to a file "Names.dat"

if ( 

inFile.fail()

 ) //test if input file exist

{

    cout << "I/O failure opening file Names.dat." << endl;



exit(1);

 //exit program if input file does not exist

}

while ( 

inFile >> name >> grade

 ) //read name and grade from input file

{

    if (grade == 7)

    {

        cout << name <<endl;//display names of grade 7’s on screen

    }

}



inFile.close();

 // close file

return 0;

}
```


Question **17**

Answer saved

Marked out of
1.00

🚩 Flag question

Choose the correct statement to correctly declare a user-defined data type that defines a pointer to a `float` variable

`typedef floatPtr *float;` ⚡Question **18**

Answer saved

Marked out of
1.00

🚩 Flag question

What is the value of `numbers.size()` after the following code?

```
vector<float> numbers;
```

```
numbers.reserve(100)
```

Select one:

- ☒ Unknown
- ☐ 0
- ☐ 100
- ☐ 10

Clear my choice

Question 19

Answer saved

Marked out of 9.00

 Remove flag

Consider the class `Circle`:

```
const double PI = 3.1415927;

class Circle
{
public:
    Circle (string name, double rad);

    double area () const;

    string getShapeName() const;

    double getRadius()const;

private:
    double radius;

    string shapeName;

};
```

(a) Derive a class `Cylinder` from the class `Circle`. Provide only the interface for the class `Cylinder`. The class `Cylinder` has an additional private double member variable `height`, as well as an additional member function `volume()`. Include an accessor for the member variable `height`. Redefine the member function `area()` for the class `Cylinder`. (6)

(b) Implement only the overloaded constructor for the class `Cylinder`. (3)



(a) Derive a class `Cylinder`

(b) Implement the overloaded constructor for the class `Cylinder`

Question 20

Answer saved

Marked out of 1.00

Flag question

Given the declarations

```
class Pet //implemented as an ADT
{
public:
    string getColour();
    string getType();
    //other member functions

private:
    string name;
    string colour;
    int age;
};

int nrPets = 30;

Pen myPets[nrPets];
```

To display the names of all the **brown** pets in the array `myPets` on the screen, we use the statement

```
for (int i = 0; i <= nrPets; i++ )
    if (myPets.getColour(i) == "brown" )
        cout << myPets.getName(i) << endl;
```

Select one:

☐ True

☐ False

Question **21**

Answer saved

Marked out of 1.00

🚩 Flag question

Given the following template function definition, which of the following is **NOT** a valid invocation of the function?

```
template <class T>

void swap(T& left, T& right)

{

    //implementation goes here, not relevant to the question

}

int int1, int2;

float flt1, flt2;

char ch1, ch2;

string s1, s2;
```

Select one:

- ☐ swap(ch1, ch2);
- ☐ swap(int1, int2);
- ☒ swap(int1, ch2);
- ☐ swap(s1,s2);

Clear my choice

Question **22**

Answer saved

Marked out of 1.00

🚩 Flag question

Which of the following statements about inheritance is true when a class is inherited `protected`?

Select one:

- ☐ The `private` members of the base class become `protected` members of the derived class.
- ☐ The derived class can directly access any member of the base class.
- ☐ The `protected` members of the base class become `private` members of the derived class.
- ☒ The `public` members of the base class become `protected` members of the derived class.

Clear my choice

Question **23**

Answer saved

Marked out of
1.00

🚩 Flag question

Both parameters of the function to overload the operator << are reference parameters.

Select one:

☒ True

☐ False

Question **24**

Answer saved

Marked out of
1.00

🚩 Flag question

Consider the following recursive function:

```
1. void print(int p)
2. {
3.     if (p==0)
4.         return;
5.     cout<< p << endl;
6.     print(p-1);
7.     return;
8. }
```

The base case for this function is

Select one:

☒ Lines 3 to 4

☐ Lines 5 to 6

☐ Line 5

☐ Line 6

[Clear my choice](#)

Question **25**

Answer saved

Marked out of
1.00

🚩 Flag question

What will be the output of the following code?

```
int *numbers = new int[5];

for (int i = 0; i <= 4; i++)

*(numbers + i) = i;

cout << numbers[2] << endl;
```

Select one:

- ☐ Five memory addresses
- ☒ 2
- ☐ 0
- ☐ 3

Clear my choice

Question **26**

Answer saved

Marked out of
1.00

🚩 Flag question

The following program fragment

```
6.char surname[];
7.surname[] = "Thabiso Ncube";
8.cout << surname << endl ;
```

gives the error messages:

```
C:\Users\sch... 6      Error: storage size of 'surname' isn't known
C:\Users\sch... 7      Error: expected primary-expression before ']' token
```

The code to correct the error is

```
6.char surname[];
7.strcpy("Thabiso Ncube", surname);
8.cout << surname << endl ;
```

Select one:

- ☐ True
- ☒ False

Question **27**

Answer saved

Marked out of
1.00

🚩 Flag question

Consider the following declaration

```
int* ptr = 0;
```

Which of the following statements is true?

Select one:

- ☐ None of the other statements is true.
- ☒ `ptr` points to the memory address 0.
- ☐ `ptr` is a dangling pointer.
- ☐ `ptr` is a null pointer.

[Clear my choice](#)

Question **28**

Answer saved

Marked out of
1.00

🚩 Flag question

The general case in a recursive function

Select one:

- ☒ is the place where the recursive calls are made
- ☐ allows the function to return without making a recursive call
- ☐ stops the process of recursive calls
- ☐ combines one or more base cases

[Clear my choice](#)